

Name: Aryan gupta
Enroll: 2024BITE030
Section: IT Even (B)

Artificial Intelligence – Module 4 Assignment

CO4: Investigate applications of AI techniques in intelligent agents, expert systems, machine learning models.

Q1 (a) Expert System for Disease Diagnosis

Expert systems work by using a knowledge base containing facts and rules about a specific field, and an inference engine that applies reasoning to these rules to find solutions.

Example: A Medical Diagnosis system.

Components of Expert System:

- **Knowledge Base:**
 - This is the core part of an expert system.
 - It stores all the facts, rules, and information about a specific domain.
 - Example: In a medical diagnosis system, it contains knowledge about diseases, symptoms, and treatments.
- **Inference Engine:**
 - This is like the brain of the system.
 - It applies logical reasoning to the knowledge base to solve problems or make decisions
 - It decides which rules to use and how to combine them to get the solution.
- **User Interface:**
 - This allows the user to interact with the expert system.
 - Users can input problems or questions and receive solutions or advice.
 - Example: A doctor enters symptoms, and the system shows possible diseases.
- **Explanation System:**
 - This explains how the system reached a particular solution .
 - It helps users understand the reasoning behind the decision.
 - Example: “The patient may have flu because of symptoms A, B, and C

Reasoning Strategies used by Inference Engine:

- **Forward Chaining** and **Backward Chaining**, which are two fundamental methods for processing information and solving problems in an expert system:
- 1. Forward Chaining
- Starts from known facts and applies rules to derive new facts until a goal is reached.
- Works like:
IF (condition) THEN (action)
- Used when all data is available, and the goal is unknown.
- Example:

- Fact: "Patient has fever and cough."
- Rule: IF patient has fever and cough → THEN patient may have flu.
- System concludes: "Patient may have flu."
- Used in: Diagnostic and monitoring systems (e.g., medical diagnosis).

2. Backward Chaining (Goal-Driven Reasoning)

Starts with a goal or hypothesis and works backward to determine what facts must be true to support it.

The system checks if there's enough evidence to prove the goal.

Example:

Goal: "Is the patient having flu?"

System searches rules that conclude flu → then checks if facts (fever, cough) are true.

- Used in: Planning and decision-making systems (e.g., expert advisors).

Q1 (b) Fuzzy Logic for a Smart Air Conditioner

Fuzzy Variables:

- Input 1 — Temperature: linguistic terms {Low, Medium, High}
- Input 2 — Humidity: linguistic terms {Low, Medium, High}
- Output — Fan Speed: linguistic terms {Slow, Medium, Fast}

Two Suitable Fuzzy Rules

- Rule 1: IF Temperature is High AND Humidity is High THEN Fan Speed is Fast. (Hot, humid air needs strong, rapid cooling and moisture removal.)
- Rule 2: IF Temperature is Low AND Humidity is Low THEN Fan Speed is Slow. (Cool, dry conditions need only gentle circulation, saving energy and avoiding over-cooling.)

(Additional rules — e.g., Medium/Medium → Medium — would normally complete the rule base, but two are illustrated here as asked.)

How Membership Functions Handle Uncertain Inputs

A real temperature or humidity reading rarely falls cleanly into one category — 29°C is 'somewhat high' and 'somewhat medium' at once. Membership functions solve this by assigning each crisp input a degree of membership (between 0 and 1) in every linguistic set, instead of forcing a hard 0/1 classification:

- Triangular/trapezoidal curves are typically used, e.g., Temperature: Low = trapezoid over 0–20°C, Medium = triangle peaking at 25°C spanning 18–32°C, High = trapezoid over 28–45°C.
- Overlapping regions mean a reading like 27°C might have membership 0.4 in 'Medium' and 0.6 in 'High' simultaneously — this fuzzification step is what lets the system respond smoothly rather than switching fan speed abruptly at an arbitrary threshold.

- Each rule fires to the degree its conditions are satisfied (using AND = minimum of the two memberships), and the outputs of all fired rules are combined and defuzzified — commonly via the centroid method — to yield one crisp fan-speed value (e.g., a specific RPM), giving smooth, human-like control instead of jarring on/off switching.

Q1 (c) Temporal Fuzzy Logic for an Industrial Process

Standard fuzzy logic evaluates only the current instantaneous reading, which is a problem when temperature is continuously changing and subject to sudden spikes: a controller reacting only to the present value can over-react to noise or under-react to a genuine sustained trend. Temporal fuzzy logic extends fuzzy logic by treating time explicitly as part of the fuzzy reasoning, using both the current value and its behaviour over a recent time window.

Key Ideas

- Time-indexed fuzzy sets: Membership functions are evaluated not just for 'Temperature' but for derived temporal variables such as Rate_of_Change (Falling, Steady, Rising, Rapidly_Rising) computed from a sliding window of recent samples.
- Fuzzy rules over trends: Rules combine the instantaneous fuzzy state with the temporal trend, e.g., IF Temperature is High AND Rate_of_Change is Rapidly_Rising THEN reduce_heater_output Sharply AND Immediately; IF Temperature is High AND Rate_of_Change is Steady THEN reduce_heater_output Gradually. This distinguishes a genuinely dangerous runaway rise from a high-but-stable state.
- Temporal smoothing/history: A short history buffer (sliding time window) is used to filter out momentary sensor spikes/noise from real sustained variations, so the controller doesn't overreact to a single transient blip.
- Predictive/anticipatory control: By extrapolating the fuzzy trend forward a short time, the controller can pre-emptively adjust actuators (e.g., start cooling slightly before a threshold is crossed) rather than only reacting after the fact — important for processes where correction has lag.

In effect, temporal fuzzy logic gives the control system 'memory,' letting it make decisions based on both where the process is and where it is heading, which produces smoother, more stable, and safer control action for a process with continuously changing values and sudden variations than a memoryless fuzzy controller would.

Q1 (d) PROLOG Knowledge Base for Family Relationships

Representing Facts

PROLOG represents knowledge declaratively as facts (base relationships asserted to be true) and rules (relationships derived from other relationships via logical conditions). For the university's system, base facts capture direct parent–child data supplied by the university's records:

```
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
```

```
parent(bob, pat).  
parent(pat, Jim).
```

Representing Derived Relationships as Rules:

Relationships such as grandparent and sibling are not stored directly — they are derived at query time from the parent/2 facts using rules:

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

```
ancestor(X, Y) :- parent(X, Y).
```

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Here, grandparent/2 reads as "X is a grandparent of Z if there exists some Y such that X is a parent of Y and Y is a parent of Z." sibling/2 finds two different people, X and Y, who share a common parent Z, with $X \neq Y$ preventing a person being called their own sibling. ancestor/2 is defined recursively to show how PROLOG can capture multi-generation relationships as well.

Deriving Relationships Using Queries

Once the facts and rules are loaded, the university can pose queries and PROLOG's inference engine (unification + SLD resolution + backtracking) automatically searches for all consistent answers:

```
?- grandparent(tom, ann).  
true.
```

```
?- sibling(ann, pat).  
true.
```

```
?- grandparent(tom, X).  
X = ann ;  
X = pat.
```

For grandparent(tom, ann), PROLOG unifies $X = \text{tom}$, $Z = \text{ann}$ and searches for a Y satisfying parent(tom, Y) and parent(Y, ann); it finds $Y = \text{bob}$, so the query succeeds. For sibling(ann, pat), it finds a common parent $Z = \text{bob}$ with $X = \text{ann}$, $Y = \text{pat}$, $X \neq Y$, so it succeeds. When a variable such as X is left open (grandparent(tom, X)), PROLOG's backtracking search returns every matching solution one at a time (ann, then pat) rather than only the first. This shows how PROLOG turns a static, small set of asserted facts into a rich network of derivable relationships the university needs to enter only direct parent links, and any relationship (sibling, grandparent, ancestor, etc.) can be queried on demand.